Procédure HAProxy - Répartition des tâches serveur web apache2

Sommaire

Sommaire.....1
Légende.....1
Informations additionnelles.....2
Partie 1 - Présentation de HAProxy.....2
Partie 2 - Prérequis.....2
Sous partie 1 - Configuration réseau.....2
Sous partie 2 - Modification des sites apache2.....4
Partie 3 - Configuration HaProxy.....6
Partie 4 -Test.....8

Légende

Le texte affiché dans cette couleur correspond à une commande.
Le texte affiché dans cette couleur correspond à un champ entré durant de processus d'une commande.
Le texte affiché dans cette couleur correspond à une modification dans un fichier donné.

Informations additionnelles

Os utilisé	3 Debian 13.1
Procédure réalisée sur machine virtuelle :	oui

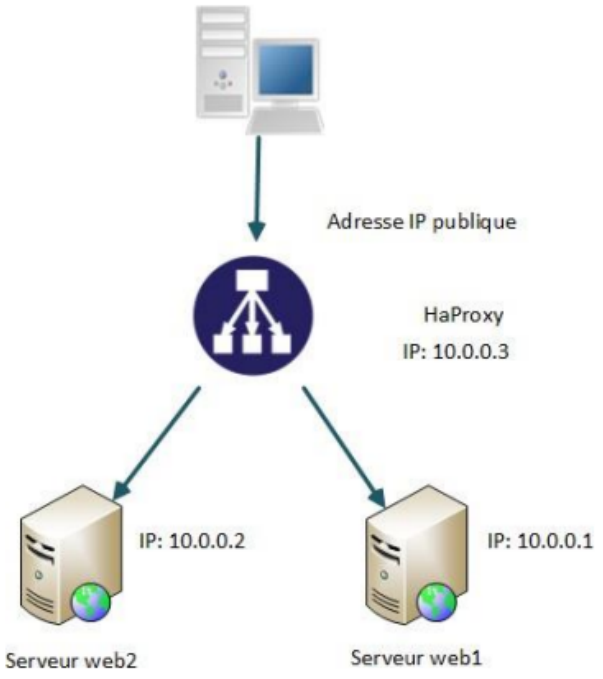
Partie 1 - Présentation de HAProxy

HAProxy (High Availability Proxy) est un logiciel open-source reconnu pour ses performances et sa fiabilité en tant que load balancer (répartiteur de charge) et proxy pour les protocoles TCP et HTTP. Il est couramment utilisé pour assurer la répartition du trafic réseau entre plusieurs serveurs backend, améliorer la disponibilité des services, et optimiser la résilience des infrastructures web. Grâce à sa configuration souple et sa faible empreinte mémoire, HAProxy s'intègre facilement dans des environnements variés, qu'ils soient simples ou à haute disponibilité.

Partie 2 - Prérequis

Sous partie 1 - Configuration réseau

Pour réaliser cette procédure nous aurons besoin d'une infrastructure réseau bien spécifique.



1) Commençons par la première machine “debian-haproxy”

   Cette Debian devra avoir la configuration suivante :

   Il faut 2 cartes réseau, sur lesquelles nous allons attribuer des adresses IP différentes. Pour consulter le nom des cartes réseau, il faudra faire cette commande.
   → dmesg | grep enp0s

```
3.146121] e1000 0000:00:08.0 enp0s8: renamed from eth1
3.147509] e1000 0000:00:03.0 enp0s3: renamed from eth0
```

   Ici nous avons enp0s3 et enp0s8.

   Dans VirtualBox enp0s3 sera en accès par pont et enp0s8 sera en réseau interne.

   Voici la configuration IP.

```
# The primary network inter
allow-hotplug enp0s3
#iface enp0s3 inet dhcp
iface enp0s3 inet static
address 172.17.219.76
netmask 255.255.0.0
gateway 172.17.0.1

iface enp0s8 inet static
address 10.0.0.3
netmask 255.255.255.0
```

2) Passons ensuite à la seconde machine “debian-web1”

La carte réseau sera configurée en réseau interne.

Avant de changer la configuration IP, installez le service apache2 avec la commande

```
apt install apache2 -y
```

La configuration IP sera la suivante.

```
# The primary network interface
allow-hotplug enp0s3
#iface enp0s3 inet dhcp
iface enp0s3 inet static
address 10.0.0.1
netmask 255.255.255.0
```

3) Passons à la troisième machine “debian-web2”

Vous pouvez tout à fait cloner “debian-web1” à condition de générer une nouvelle adresse mac.

Avant de changer la configuration IP, installez le service apache2 avec la commande

```
apt install apache2 -y
```

La configuration IP sera la suivante.

```
# The primary network interface
allow-hotplug enp0s3
#iface enp0s3 inet dhcp
iface enp0s3 inet static
address 10.0.0.2
network 255.255.255.0
```

Vous avez à présent terminé la configuration réseau.

Sous partie 2 - Modification des sites apache2

Sur les machines “debian-web1” et “debian-web2” il faudra juste procéder à quelques modifications pour les futurs tests.

Il faudra changer les pages HTML.

→ Sur le serveur 1 :

```
nano /var/www/html/index.html
```

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Serveur 1</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f2f2f2;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
        }
        h1 {
            color: #333;
        }
    </style>
</head>
<body>
    <h1>Bienvenue sur le serveur 1</h1>
</body>
</html>
```

Puis sur le serveur 2 :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Serveur 2</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f2f2f2;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
```

```
            margin: 0;
        }
        h1 {
            color: #333;
        }
    </style>
</head>
<body>

<h1>Bienvenue sur le serveur 2</h1>
</body>
</html>
```

Cela permet à chaques sites d'avoir le bon affichage et de savoir quel serveur nous utilisons.

Partie 3 - Configuration HaProxy

Il faut commencer par installer Haproxy

```
apt install haproxy
```

Il faudra ensuite éditer le fichier de configuration

```
nano /etc/haproxy/haproxy.cfg
```

```
global
        log /dev/log    local0
        log /dev/log    local1 notice
        chroot /var/lib/haproxy
        stats socket /run/haproxy/admin.sock mode 660 level admin
        stats timeout 30s
        user haproxy
        group haproxy
        daemon
```

```
        # Default SSL material locations
        ca-base /etc/ssl/certs
        crt-base /etc/ssl/private

        # See:
https://ssl-config.mozilla.org/#server=haproxy&server-version=2.0.3&conf
ig=intermediate
        ssl-default-bind-ciphers
ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AE
S256-GCM-SH>
        ssl-default-bind-ciphersuites
TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA2
56
        ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

defaults
        log     global
        mode    http
        option  httplog
        option  dontlognull
        timeout connect 5000
        timeout client  50000
        timeout server  50000
        errorfile 400 /etc/haproxy/errors/400.http
        errorfile 403 /etc/haproxy/errors/403.http
        errorfile 408 /etc/haproxy/errors/408.http
        errorfile 500 /etc/haproxy/errors/500.http
        errorfile 502 /etc/haproxy/errors/502.http
        errorfile 503 /etc/haproxy/errors/503.http
        errorfile 504 /etc/haproxy/errors/504.http

frontend proxypublic
        bind 172.17.219.76:80
        default_backend fermeweb

backend fermeweb
        balance roundrobin
        option httpclose
        option httpchk HEAD / HTTP/1.0
        server debian-web1 10.0.0.1:80 check
        server debian-web2 10.0.0.2:80 check
        stats uri /statsHaproxy
        stats auth admin:admin
        stats admin if TRUE
        stats refresh 30s
```

Il faut bien prendre soin de remplacer L'ip public ainsi que le nom des 2 machines serveurs.

ensuite relancer le service avec

```
systemctl restart haproxy
```

Partie 4 -Test

Entrez l'ip de la machine 1 et en rafraichissant la page on peut voir qu'on change de serveur à chaque requête

http://172.17.219.76/statsHaproxy

ici c'est les stats